ASSIGNMENT 05 – PFP191

Duration: 70'

==========

Software Requirements

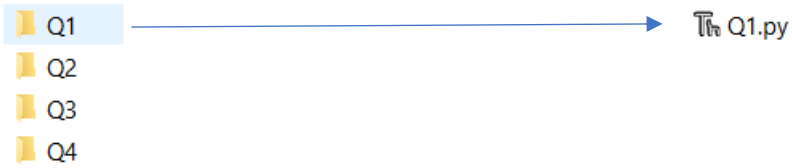
- Thonny, Notepad, Command Prompt, WinRAR / WinZip (or compress to ZIP file of Windows), Windows Explorer (File Explorer) on Windows 7 and above.

Students are ONLY Allowed to use:

- The materials like sample codes and program examples are stored on computer only.

Instructions

- Step 1: Students download the given materials from PEA Client.
- Step 2: Students read questions and prepare answers in the given template.
- Step 3: Submit a solution for each question:
  - The result is one folder for each question (Q1,Q2,Q3,Q4…), which contains **.py** source files.
    Example: Folder "Q1" contains Q1.py for question 1



  - For each question, you must submit one folder as above to PEA Client.

❖ **Importance:**

- Solutions will be marked by Automated Marking Software.
- **The use of tools other than those allowed in the above section whether intentionally or unintentionally, is considered a violation of the exam rules, and the mark is 0**
- Do not change the names of the folders, files, and, files and structure (format) of .py files specified in the exam. If you change it, the marking software can not find the execute file (.py) or the program output to mark, thus the score will be 0
- Do not edit given statements in the **main** function. If you change, the marking software can not mark and the score will be 0.

==========

Question 1: (2 marks)

The given Q1.py file contains statements about input and output data of objects from the Student class. You should write the statements to complete the Student class as described below:

Student
- code : string - name: string - markOfMath : int - markOfEng: int

Where:

- Student (code:string, name:string, department: String, markOfMath : int, markOfEng: int) -

+ Student (code:string, name:string, markOfMath : int, markOfEng: int) + getMarkOfMath():int + getMarkOfEng():int + getGPA(): float + toString(): string

parameterized constructor: set the values to the code, name and department.

- getMarkOfMath():int - return markOfMath
- getMarkOfEng():int - return markOfEng
- getGPA():float – return the GPA of Student with formular: gpa = (markOfMath *3 + markOfEng *2)/5
- toString():string – return a string format that contains all the information of the **Student**: ***code, name, markOfMath, markOfEng***. The ***markOfMath*** *and* ***markOfEng*** are formatted with ***two decimal places***.

Notes:

- You can create new function(s) if you see it is necessary.

- Do not edit given statements in the **main** function.

Sample input and output:

Enter student code: SE200123 Enter student name: Jack Enter mark of Math: 7 Enter mark of English: 8 Student information: SE200123,Jack,7,8 1. Test getMark() 2. Test getGPA() Enter TC(1/2):1 Output for marking: **OUTPUT:** 7,8	Enter student code: SE200123 Enter student name: Jack Enter mark of Math: 7 Enter mark of English: 8 Student information: SE200123,Jack,7,8 1. Test getMark() 2. Test getGPA() Enter TC(1/2):2 Output for marking: **OUTPUT:** 7.40

Question 2: (3 marks)

The given Q2.py file contains statements about input and output data of objects from the **BankAccount** class. You should write the statements to complete the **BankAccount** class as described below:

BankAccount
- account: string - holder: string - balance: float
+ BankAccount (account: string, holder: string, balance: float)

Where:

- BankAccount (account: string, holder: string, balance: float) - parameterized constructor: set the values to the account, holder and balance.

+ deposit(float amount) + withdraw(float amount): bool + toString(): string

- deposit(float amount) - Increase balance with amount
- withdraw(float amount): bool – if balance < amount return false; otherwise decrease balance with amount and return true
- toString():string – return a string format that contains all the information of the BankAccount: ***account, holder and balance***. The ***balance*** is formatted with ***two decimal places***.

Notes:

- You can create new function(s) if you see it is necessary.

- Do not edit given statements in the **main** function.

Sample input and output:

Enter account number: 123456 Enter account holder name: Jack Enter initial balance: 2000 Account information: 123456,Jack,2000.00 1. Test deposit() 2. Test withdraw() Enter TC(1/2):1 Enter amount to deposit: 500 Output for marking: **OUTPUT:** 123456,Jack,2500.00	Enter account number: 123456 Enter account holder name: Jack Enter initial balance: 2000 Account information: 123456,Jack,2000.00 1. Test deposit() 2. Test withdraw() Enter TC(1/2):2 Enter amount to withdraw: 500 Output for marking: **OUTPUT:** 123456,Jack,1500.00	Enter account number: 123456 Enter account holder name: Jack Enter initial balance: 2000 Account information: 123456,Jack,2000.00 1. Test deposit() 2. Test withdraw() Enter TC(1/2):2 Enter amount to withdraw: 2500 Output for marking: **OUTPUT:** 123456,Jack,2000.00

Question 3: (2 marks)

The given Q3.py file contains statements about input and output data of objects from the **Product** class. You should write the statements to complete the **Product** class as described below:

Product
- id: string - name: string - price: int - quantity: int
+ Product(id: string, name: string, price: int, quantity: int) + getTotal(): int

Where:

- Product(id: string, name: string, price: int, quantity: int) - parameterized constructor: set the values to the id, name, price and quantity.
- getTotal(): int – return price * quantity, if quantity >=50, discount 10%

+ toString(): string

- toString():string – return a string format that contains all the information of the Product with format: **id, name, price, quantity**.

Notes:

- You can create new function(s) if you see it is necessary.

- Do not edit given statements in the **main** function.

Sample input and output:

Enter Product id: P001 Enter Product name: RamDDR4 Enter Product price: 100 Enter Product quantity: 50 1. Test getTotal() 2. Test constructor() Enter TC(1/2):1 Output for marking: **OUTPUT:** 4500.00	Enter Product id: P001 Enter Product name: RamDDR4 Enter Product price: 100 Enter Product quantity: 40 1. Test getTotal() 2. Test constructor() Enter TC(1/2):1 Output for marking: **OUTPUT:** 4000.00	Enter Product id: P001 Enter Product name: Ram Enter Product price: 100 Enter Product quantity: 50 1. Test getTotal() 2. Test constructor() Enter TC(1/2):2 Output for marking: **OUTPUT:** P001,Ram,100,50

Question 4: (3 marks)

The given Q4.py file contains statements about input and output data of objects from the **Employee** class. You should write the statements to complete the **Employee** class as described below:

Employee
- id: string - name: string - salary: float - position: string
+ Employee(id: string, name: string, salary: float, position: string) + getAllowance(): float + getIncome(): float + toString(): string

Where:

- Employee(id: string, name: string, salary: float, position: string) - parameterized constructor: set the values to the id, name, price and quantity.
- getAllowance(): float - returns the allowance according to the job position. If the position is :
  - DEV, the allowance is 20% of salary
  - TES is 10% of salary
  - Other positions are 0.
- getIncome(): float – return salary + allowance
- toString():string – return a string format that contains all the information of the Product with format: **id, name, salary, position, income**.

Notes:

- You can create new function(s) if you see it is necessary.

- Do not edit given statements in the **main** function.

Sample input and output:

Enter Employee id: E001 Enter Employee name: Jack Enter Employee salary: 2000 Enter Employee position: DEV 1. Test getAllowance() 2. Test getIncome() Enter TC(1/2):1 Output for marking: **OUTPUT:** 400.00	Enter Employee id: E001 Enter Employee name: Jack Enter Employee salary: 2000 Enter Employee position: TES 1. Test getAllowance() 2. Test getIncome() Enter TC(1/2):1 Output for marking: **OUTPUT:** 200.00	Enter Employee id: E001 Enter Employee name: Jack Enter Employee salary: 2000 Enter Employee position: HR 1. Test getAllowance() 2. Test getIncome() Enter TC(1/2):2 Output for marking: **OUTPUT:** 2000.00